

SQL Data Analysis Project: Extracting Insights with SQL queries on Pizza Sales



A Portfolio Project
Mohit Kumar
12/4/2025

Project Overview:

Objective:-

- Use SQL to extract and analyze data on pizza company from a csv uploaded file on a MySQL database.

Key Tasks:-

- Import CSV data of a Pizza company on MySQL Database.
- Write SQL queries to answer specific business questions.
- Display and interpret query outputs.

Tools Used:-

- MySQL, CSV from a Data Source.

DATA SOURCE AND DATABASE SETUP

Content:-

Data Source:- CSV file containing sales data, pizza category, pizza type, size ordered, order by weekdays, hours, date, customer data.

Setup Steps:-

Uploaded CSV file into SQL Database using MySQL Workbench.

Created a table with appropriate Schema with defined columns like pizza_id, Pizza_type_id, Order_id, Product type, By time, Weekday.

Verified Data integrity by checking for duplicate and null values.

SQL Query 1: Retrieve the Total Number of orders Placed

-- Retrieve the total number of orders placed.

```
select count(order_id) as total_orders from orders;
```

Result Grid			 Filter Rows:
	total_orders		
▶	21350		

SQL Query 2: Identify the Highest Priced Pizza

```
-- Identify the highest-priced pizza.
```

```
select pizza_types.name, pizzas.price  
from pizza_types join pizzas  
on pizza_types.pizza_type_id = pizzas.pizza_type_id  
order by pizzas.price desc limit 1;
```

Result Grid		 Filter Rows:
	name	price
▶	The Greek Pizza	35.95

SQL Query 3: Identify the most common pizza size Ordered.

```
-- Identify the most common pizza size ordered.
```

```
select pizzas.size, count(orders_details.order_details_id) as order_count
from pizzas join orders_details
on pizzas.pizza_id = orders_details.pizza_id
group by pizzas.size order by order_count desc limit 1;
```

Result Grid   Filter Rows:		
	size	order_count
▶	L	18526

SQL Query 4: List the top 5 most ordered Pizza types along with their quantities.

```
-- List the top 5 most ordered pizza types along with their quantities.

select pizza_types.name, sum(orders_details.quantity) as quantity
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join orders_details
on orders_details.pizza_id = pizzas.pizza_id
group by pizza_types.name order by quantity desc limit 5;
```

Result Grid   Filter Rows:		
	name	quantity
▶	The Classic Deluxe Pizza	2453
	The Barbecue Chicken Pizza	2432
	The Hawaiian Pizza	2422
	The Pepperoni Pizza	2418
	The Thai Chicken Pizza	2371

SQL Query 5: Join the necessary tables to find the total quantity of each pizza category ordered.

```
-- Join the necessary tables to find the total quantity of each pizza category ordered.
```

```
select pizza_types.category,  
sum(orders_details.quantity) as quantity  
from pizza_types join pizzas  
on pizza_types.pizza_type_id = pizzas.pizza_type_id  
join orders_details  
on orders_details.pizza_id = pizzas.pizza_id  
group by pizza_types.category order by quantity desc;
```

Result Grid			Filter Rows:
	category	quantity	
▶	Classic	14888	
	Supreme	11987	
	Veggie	11649	
	Chicken	11050	

SQL Query 6: Determine the distribution of orders by hours of day.

```
-- Determine the distribution of orders by hour of the day.
```

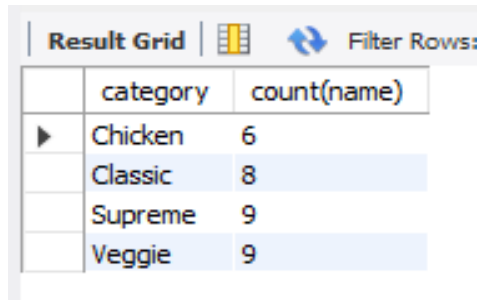
```
select hour(order_time) as hour, count(order_id) as order_count  
from orders  
group by hour(order_time);
```

Result Grid				 Filter Rows
	hour	order_count		
▶	11	1231		
	12	2520		
	13	2455		
	14	1472		
	15	1468		
	16	1920		
	17	2336		

SQL Query 7: Join the relevant tables to find the category-wise distribution of pizzas.

```
-- Join relevant tables to find the category-wise distribution of pizzas.
```

```
select category, count(name) from pizza_types  
group by category;
```



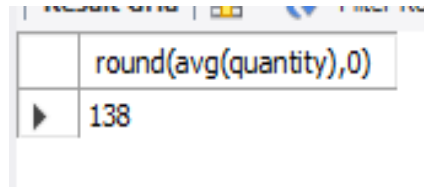
The screenshot shows a database interface with a 'Result Grid' tab. It contains a table with two columns: 'category' and 'count(name)'. The table has four rows of data: 'Chicken' with a count of 6, 'Classic' with a count of 8, 'Supreme' with a count of 9, and 'Veggie' with a count of 9. A 'Filter Rows' button is visible at the top right of the grid.

	category	count(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9

SQL Query 8: Group the orders by date and find the average no of pizzas ordered per day.

```
-- Group the orders by date and calculate the average number of pizzas ordered per day.
```

```
select round(avg(quantity),0) from  
(select orders.order_date, sum(orders_details.quantity) as quantity  
from orders join orders_details  
on orders.order_id = orders_details.order_id  
group by orders.order_date) as order_quantity;
```



round(avg(quantity),0)	
▶	138

SQL Query 9: Determine the top 3 most ordered pizzas type based on revenue.

```
-- Determine the top 3 most ordered pizza types based on revenue.
```

```
select pizza_types.name, sum(orders_details.quantity * pizzas.price) as revenue
from pizza_types join pizzas
on pizzas.pizza_type_id = pizza_types.pizza_type_id
join orders_details
on orders_details.pizza_id = pizzas.pizza_id
group by pizza_types.name order by revenue desc limit 3;
```

	name	revenue
▶	The Thai Chicken Pizza	43434.25
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41409.5

SQL Query 10: Calculate the percentage contribution of each pizza type to total revenue.

```
-- Calculate the percentage contribution of each pizza type to total revenue.  
  
select pizza_types.category, round(sum(orders_details.quantity * pizzas.price) /  
(select round(sum(orders_details.quantity * pizzas.price),2) as total_sales  
from orders_details join pizzas  
on pizzas.pizza_id = orders_details.pizza_id),2) * 100 as revenue  
from pizza_types join pizzas  
on pizza_types.pizza_type_id = pizzas.pizza_type_id  
join orders_details  
on orders_details.pizza_id = pizzas.pizza_id  
group by pizza_types.category order by revenue desc;
```

	category	revenue
▶	Classic	27
	Supreme	25
	Veggie	24
	Chicken	24

SQL Query 11: Analyze the Cumulative revenue generated over time.

```
-- Analyze the cumulative revenue generated over time.
```

```
select order_date, sum(revenue) over (order by order_date) as cummulative_revenue  
from
```

```
(select orders.order_date, sum(orders_details.quantity * pizzas.price) as revenue  
from orders_details join pizzas  
on orders_details.pizza_id = pizzas.pizza_id  
join orders  
on orders.order_id = orders_details.order_id  
group by orders.order_date) as sales;
```

Result Grid   Filter Rows:		
	order_date	cummulative_revenue
▶	2015-01-01	2713.8500000000004
	2015-01-02	5445.75
	2015-01-03	8108.15
	2015-01-04	9863.6
	2015-01-05	11929.55

SQL Query 12: Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
-- Determine the top 3 most ordered pizza types based on revenue for each pizza category.
```

```
select name, revenue from
```

```
(select category, name, revenue,  
rank() over(partition by category order by revenue desc) as rn  
from
```

```
(select pizza_types.category, pizza_types.name, sum(orders_details.quantity * pizzas.price) as revenue  
from pizza_types join pizzas  
on pizza_types.pizza_type_id = pizzas.pizza_type_id  
join orders_details  
on orders_details.pizza_id = pizzas.pizza_id  
group by pizza_types.category, pizza_types.name) as a) as b  
where rn<=3;
```

Result Grid			Filter Rows:
	name	revenue	
▶	The Thai Chicken Pizza	43434.25	
	The Barbecue Chicken Pizza	42768	
	The California Chicken Pizza	41409.5	
	The Classic Deluxe Pizza	38180.5	
	The Hawaiian Pizza	32273.25	
	The Pepperoni Pizza	30161.75	
	The Spicy Italian Pizza	34831.25	
	The Italian Supreme Pizza	33476.75	
	The Sicilian Pizza	30940.5	
	The Four Cheese Pizza	32265.70000000065	
	The Mexicana Pizza	26780.75	
	The Five Cheese Pizza	26066.5	

Challenges and Solutions:

- Challenge 1: Larger Dataset slowed down Query Performance.
- Solution: Added indexes on frequently queried column like transaction_date.
- Challenge 2: Missing values in some column.
- Solution: Filtered out null values in queries.

Conclusions and Next Steps

Conclusions:

- This Project demonstrates my proficiency in SQL for data extractions, analysis and interpretation.

Skills Showcased:

- Importing CSV data into SQL Databases.
- Writing complex SQL queries like aggregation, grouping and sorting.
- Visualizing and interpreting Query outputs.

Next Steps;

- Optimize queries for Larger Datasets using Advanced Indexing.
- Integrate SQL with BI tool for Interactive Dashboards.